

Multi-layer perceptrons

Neurons, XOR and the hidden layer, activations, and the forward pass as matrix products.

THE QUESTION

One neuron draws one straight boundary. How do we get curved boundaries, and how do we train all the weights?

THE INTUITION

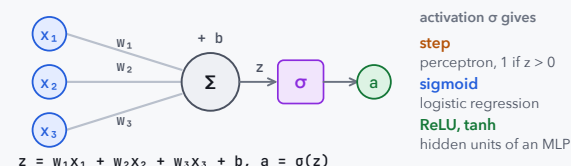
Put layers of neurons between input and output. Each hidden layer is a linear map followed by a nonlinear activation, which moves the points into a new space where a line can split them. Gradient descent trains every weight; the chain rule, applied layer by layer, gives the gradients.

KEY TAKEAWAY

$h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)})$. Without σ the layers collapse into one linear map. With it, one wide enough hidden layer can approximate any continuous function, and backpropagation gives the gradients.

input $x \rightarrow z = Wh + b \rightarrow h = \sigma(z) \rightarrow$ repeat per layer $\rightarrow \hat{y}$ and the loss $\rightarrow$ backprop, step

1 A neuron sums, then activates



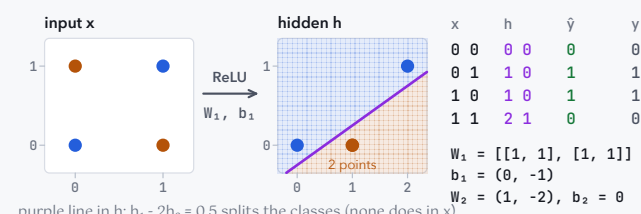
$z = w^T x + b$, then $a = \sigma(z)$. The activation decides the model: a step gives the perceptron, a sigmoid gives logistic regression (the logistic regression sheet is one neuron). Hidden units of an MLP use ReLU or tanh.

2 One line cannot split XOR



The perceptron notebook's rule $w \leftarrow w + \eta(y - \hat{y})x$ ($\eta = 0.01$, 100 passes) finds lines for AND and OR. For XOR it ends at $w = (0.01, -0.01, 0)$ and gets 2 of 4 right; no line puts $(0, 1)$, $(1, 0)$ on one side and $(0, 0)$, $(1, 1)$ on the other. A hand-made feature x_1x_2 fixes it ($w = (0, 0.01, 0.01, -0.04)$, 4 of 4). A hidden layer learns such features.

3 A hidden layer makes XOR separable



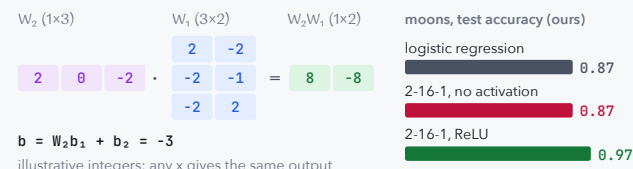
The vectorisation notebook's 2-2-1 net: $h = \text{ReLU}(W_1x + b_1)$, $\hat{y} = W_2h + b_2$. The hidden layer sends $(0, 1)$ and $(1, 0)$ to the same point $(1, 0)$ and $(1, 1)$ to $(2, 1)$. There the line $h_1 - 2h_2 = 0.5$ separates the classes, and $\hat{y} = h_1 - 2h_2$ gives 0, 1, 1, 0 exactly.

4 Activations and their slopes



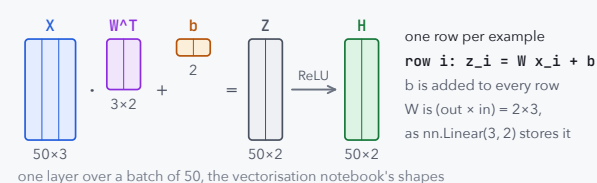
$\sigma(z) = 1/(1 + e^{-z})$ with $\sigma' = \sigma(1 - \sigma)$, in $(0, 1)$. $\tanh z = (e^z - e^{-z})/(e^z + e^{-z})$ with slope $1 - \tanh^2 z$, in $(-1, 1)$. $\text{ReLU}(z) = \max(0, z)$ with slope 0 or 1: cheap, and it does not shrink gradients where it is active, the reason it took over hidden layers. Softmax turns the last layer's scores into class probabilities.

5 Without an activation, layers collapse



$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$: any stack of linear layers is one linear map. Our 2-16-1 net without activations, trained on the moons, ends at the logistic regression weights $(0.628, 1.198, -4.577)$ to three decimals and the same 0.87 test accuracy. With ReLU in between it reaches 0.97.

6 The forward pass is matrix products



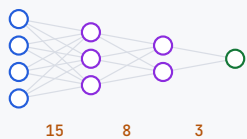
Per example, layer l computes $z^{(l)} = W^{(l)}h^{(l-1)} + b^{(l)}$ and $h^{(l)} = \sigma(z^{(l)})$, from $h^{(0)} = x$ to $\hat{y} = h^{(L)}$. $W^{(l)}$ is $n_l \times n_{l-1}$. For N examples stacked as rows, $H^{(l)} = \sigma(H^{(l-1)}W^{(l)T} + b^{(l)T})$, $N \times n_l$. The tutorial's $x = (1, 2)$ through rows $(0.5, 1)$ and $(-0.3, 0.8)$, $b = (0.1, -0.2)$ and ReLU gives $(2.6, 1.1)$.

Parameters, backprop and what one layer can fit

Counting weights, the backward pass, a NumPy loop, a trained boundary, universal approximation, depth.

7 Each layer has $n_{\text{in}}n_{\text{out}} + n_{\text{out}}$ parameters

4-3-2-1: $12+3, 6+2, 2+1 = 26$



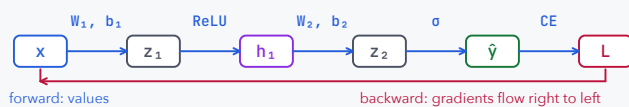
total parameters (log scale)

4-3-2-1	26
4-3-1	19
10-5-2	67
10-50-2	652
784-128-10	101,770

784-128-10 is our example

A weight per connection plus a bias per output unit: $15 + 8 + 3 = 26$ for the notebook's 4-3-2-1 net, 19 for the tutorial's 4-3-1. With 10 inputs and 2 outputs, 5 or 50 hidden units give 67 or 652: width adds capacity, and overfitting risk, fast.

8 Backprop: the chain rule, layer by layer



forward: values

$\delta_2 = \hat{y} - y$ (sigmoid + cross-entropy)
 $\delta_1 = (W_2^T \delta_2) * \text{ReLU}'(z_1)$, * elementwise
 $\text{grad } W_2 = \delta_2 h_1^T$, $\text{grad } W_1 = \delta_1 x^T$

backward: gradients flow right to left

Let $\delta^{(l)} = \partial L / \partial z^{(l)}$. A sigmoid output with cross-entropy has $\delta^{(L)} = \hat{y} - y$, as in logistic regression. Going back:

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot \sigma'(z^{(l)}),$$

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} h^{(l-1)T}, \quad \frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}.$$

On our 13-parameter net these match finite differences to 6×10^{-9} . The general machinery: the automatic differentiation sheet.

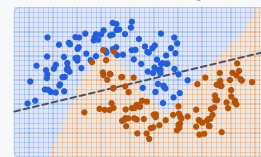
9 The whole training loop in NumPy

Full-batch gradient descent on mean cross-entropy for a 2-16-1 net, from seeded random weights (W_1 entries from $\mathcal{N}(0, 1)$):

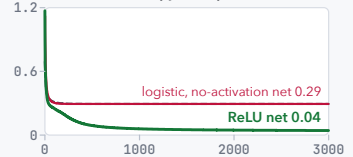
```
for step in range(3000):
    Z1 = X @ W1.T + b1          # N x 16
    H1 = np.maximum(Z1, 0)      # ReLU
    p = sig(H1 @ W2.T + b2)     # N x 1
    d2 = (p - y) / N           # dL/dz2
    gW2, gb2 = d2.T @ H1, d2.sum(0)
    d1 = (d2 @ W2) * (Z1 > 0)  # dL/dz1
    gW1, gb1 = d1.T @ X, d1.sum(0)
    W1 -= lr * gW1; b1 -= lr * gb1
    W2 -= lr * gW2; b2 -= lr * gb2
```

10 A trained MLP bends its boundary

2-16-1 ReLU net (shade), logistic (dashed)



mean cross-entropy vs step

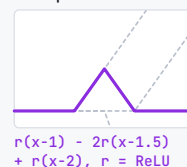


200 training moons (ours, seed 0) · test accuracy 0.97 vs 0.87 (logistic)

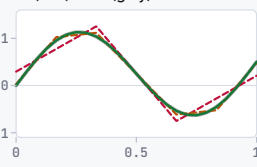
Our 200 noisy moons, $\eta = 0.5$, 3000 steps. The ReLU net's loss falls from 1.17 to 0.25 in 100 steps and 0.043 by the end: 98% train, 97% on 1000 fresh points. Logistic regression and the net without activations stop at 0.29, with a straight boundary and 87%.

11 Sums of ReLUs fit any continuous curve

a bump from 3 ReLUs



$\sin(2\pi x) + x/2$ (grey)



ReLU's	max err
3	0.293
6	0.081
24	0.006

knots evenly spaced (ours)

Three ReLUs make a bump; a weighted sum of shifted ReLUs is a piecewise-linear curve with a kink at each unit. With the output weights fitted by least squares, the worst error on our curve falls from 0.29 (3 units) to 0.081 (6) to 0.006 (24). That is the universal approximation idea. It says the weights exist, not that gradient descent finds them, and the width it needs can be large.

12 Depth multiplies derivatives

gradient factor after L layers



Backprop multiplies one σ' per layer. Sigmoid's is at most 0.25, so 5 layers scale gradients by at most $0.25^5 \approx 10^{-3}$ (the tutorial's 0.1 per layer gives 10^{-5}): early layers barely learn. ReLU passes 1 where active. Equal starting weights give every hidden unit the same gradient, so they stay copies; random starts break the tie.

13 Pause and think

Why a hidden layer? One line cannot split XOR.

Logistic regression? One sigmoid neuron.

Drop the activation? One linear map remains.

δ at a sigmoid, cross-entropy output? $\hat{y} - y$.

Slope of $(x^2 + 1)^3$ at 2? $3 \cdot 5^2 \cdot 4 = 300$.

All-zero start with ReLU? Every hidden gradient is 0.

neuron $\sigma(w^T x + b)$ · hidden layers make features · σ between layers · backprop = chain rule · random start

Before: the precision-recall curves sheet; next: the automatic differentiation sheet. Source: lecture notes "Multi-layer perceptrons", the perceptron and neural-network vectorisation notebooks and the neural-networks tutorial (nipunbatra.github.io/ml-teaching). The moons, the 1-D curve and the integer matrices are ours, seeded. Corrected: the tutorial's output-layer $\delta^{(L)}$ must be the gradient with respect to $z^{(L)}$, not $h^{(L)}$.